MimeSys Algorithms

This document describes the simulation algorithms used by MimeSys in detail. The goal is to make the data-generation mechanism transparent enough that each simulation setting can be traced back to a specific statistical operation.

The key separation is:

- Null feature generation creates simulated multi-omics matrices and their covariance/correlation structure.
- Outcome generation creates the phenotype/response after the simulated omics matrices exist.
- Microbiome two-stage emission changes how sparse microbiome-like features are emitted, but it is still part of null feature generation.

The most important conceptual point is:

$$\text{crosstalk controls cross} - \text{view feature dependence}$$

$$\text{outcome generation controls what predicts Y}$$

High crosstalk means the omics layers share more latent structure. It does not automatically mean that fusion must outperform single-layer or simpler models. Fusion performance depends on how the outcome is generated and whether signal is complementary, shared, redundant, nonlinear, or concentrated in one layer.

1. Notation

Assume M omics views. For view m:

$$X_m \in \mathbb{R}^{n \times p_m}$$

Where:

- n is the simulated sample size.
- p_m is the number of simulated features in view m.
- X_m[i, j] is feature j in view m for sample i.

The complete simulated feature matrix is:

$$X = [X_1, X_2, \ldots, X_M]$$

MimeSys fits a reusable template from empirical data. The template stores:

- empirical feature marginals,
- normal-score transformed features,
- within-view factor loadings,
- within-view factor scores,
- shared cross-view latent factors,
- mappings from shared latent factors into each view,
- optional two-stage emission parameters for sparse microbiome-like views.

2. Fitting the Template

2.1 Input

The input is a named list of matched omics matrices:

```
template <- list(
  Immunome = immunome_matrix,
  Metabolome = metabolome_matrix,
  Proteome = proteome_matrix
)
```

Rows are samples. Columns are features.

2.2 Sample matching

If row names are available, MimeSys intersects sample IDs across views and keeps only samples shared by all views.

If row names are not available, all views must have the same number of rows. In that case, rows are assumed to already be matched by position.

2.3 Optional view transformation

Before fitting, views can be transformed using view.transform.

Supported choices include:

- "none": no transformation,
- "log1p": apply log(1 + x),
- "clr": centered log-ratio style transformation for compositional data.

This step is useful for microbiome or highly skewed abundance data.

2.4 Missing values

Missing feature values are median-imputed within each feature. If feature j has missing values, those entries are replaced by:

$$\text{median}(x_j,\ \text{na.rm}\ =\ \text{TRUE})$$

2.5 Constant features

Features with no variation are removed. A constant feature cannot contribute to correlation, factor structure, or outcome signal.

2.6 Empirical quantile functions

For each feature, MimeSys stores the sorted empirical values:

$$Q_j$$

This stored empirical quantile function is later used to map simulated normal-score values back to the empirical scale.

For integer-like features, MimeSys records that the feature is discrete. Discrete features use step-function quantile mapping so that simulated values remain integer-like.

2.7 Normal-score transformation

Each empirical feature is converted to normal scores:

$$z_{ij} = \Phi^{-1}\left(\frac{\text{rank}(x_{ij})}{n_0 + 1}\right)$$

Where:

- n0 is the template sample size,
- Phi^{-1} is the standard normal quantile function,
- randomized tie handling is used to deal with zeros and repeated values.

The normal-score matrix for view m is:

$$Z_m$$

2.8 Within-view factor model

For each view, MimeSys fits a low-rank factor model on Z_m:

$$Z_m \approx F_m \Lambda_m^T + \varepsilon_m$$

Where:

- F_m is the sample-by-factor score matrix for view m,
- Lambda_m is the feature-by-factor loading matrix,
- k_m is the number of within-view factors.

This is obtained through truncated SVD.

2.9 How within-view rank is chosen

MimeSys does not use 2 log(p) by default.

If the user supplies:

```
rank = K
```

then K factors are used per view, or the supplied vector is recycled/matched across views.

If:

```
rank = NULL
```

MimeSys chooses the smallest rank explaining at least var.explained of the normal-score variance:

$$\text{k_m} \;=\; \text{smallest k such that cumulative variance explained} \;>=\; \text{var.explained}$$

Then this is capped by:

$$\text{max.rank} \;=\; \text{floor(n0 / 3)}$$

unless the user supplies a different max.rank.

Therefore:

$$k_m = \min(\text{rank needed for var. explained}, \text{max. rank}, p_m - 1)$$

2.10 Loading stabilization

For each feature:

$$h_j = ||Lambda_m[j,]||^2$$

If:

$$h_j > 0.99,$$

MimeSys shrinks that loading row so the residual variance remains positive.

This prevents invalid residual variance when simulating new normal-score data.

2.11 Shared cross-view latent space

After fitting each view, MimeSys stacks the within-view factor scores:

$$F_{\text{all}} = [F_1, F_2, \ldots, F_M]$$

It computes:

$$\text{cor}(F_{\text{all}})$$

and performs an eigen decomposition. The leading eigenvectors define the shared cross-view latent structure, similar to a MAXVAR/generalized CCA representation.

2.12 How shared rank is chosen

MimeSys does not use 2 log(p) for the shared rank either.

If the user supplies:

```
shared.rank = r
```

then MimeSys uses exactly r shared factors.

If:

```
shared.rank = NULL
```

MimeSys estimates the number of shared factors by permutation-based parallel analysis:

1. Compute observed eigenvalues of cor(F_all).
2. Independently permute sample labels within each view's factor score matrix.

3. Recompute eigenvalues from the permuted data.
4. Repeat this nperm times.
5. For each eigenvalue index, compute the 95th percentile of the permuted/null eigenvalues.
6. Keep leading observed eigenvalues that exceed the corresponding null threshold.

If the first observed eigenvalue is not above the null threshold, then:

$$\text{shared.rank} = 0$$

2.13 Shared factor scores

If shared.rank > 0, MimeSys constructs:

$$S \in \mathbb{R}^{n_0 \times r}$$

This is obtained by projecting the stacked factor scores onto the leading shared eigenvectors, then orthonormalizing and scaling the scores.

2.14 View-specific shared loading matrices

For each view:

$$B_m = \frac{F_m^T S}{n_0 - 1}$$

B_m maps the shared latent factors into view m's within-view factor space.

The template therefore contains:

$$\Lambda_m : \text{feature loadings within view } m$$

$$F_m : \text{within-view factor scores}$$

$$S : \text{shared latent scores}$$

$$B_m : \text{map from shared factors to view } m$$

$$Q_j : \text{empirical feature quantile functions}$$

3. Null Multi-Omics Feature Generation

3.1 Goal

Generate synthetic omics matrices that preserve:

- empirical marginal distributions,
- within-view dependence,
- cross-view dependence,
- optional sparse/zero-inflated microbiome behavior.

This is where crosstalk operates.

3.2 Feature count handling

For each view, requested feature count p_m can be:

- NULL: keep the template feature count,
- a scalar: same feature count for every view,
- a vector: view-specific feature counts.

3.3 If requested features are fewer than template features

If:

$$p_m <= p0_m,$$

MimeSys selects parent features from the template.

Selection can be:

- "random": random subset,
- "variance": highest-variance features first.

The selected parent feature supplies:

- empirical marginal distribution,
- loading vector,
- two-stage emission parameters if relevant.

3.4 If requested features exceed template features

If:

$$p_m > p0_m,$$

MimeSys creates offspring features.

Each offspring feature copies a parent marginal distribution but receives perturbed loadings:

$$\Lambda_{\text{offspring}} = \sqrt{1 - \text{feature.jitter}}\,\Lambda_{\text{parent}} + \sqrt{\text{feature.jitter}}\,\Lambda_{\text{other}}$$

Interpretation:

- small feature.jitter: offspring closely resembles the parent,
- large feature.jitter: offspring is more perturbed.

3.5 Crosstalk

For view m, crosstalk is:

$$\gamma_m$$

It controls how strongly the simulated within-view factor scores are driven by the shared latent factors.

In the factor simulator:

$$f_m = \sqrt{\gamma_m}\, S_{\text{new}} B_m^T$$

$$+E_m \sqrt{I - \gamma_m B_m B_m^T}$$

Where:

- f_m is the simulated factor score matrix for view m,
- S_new is the simulated shared latent matrix,
- B_m is the template-estimated map from shared factors to view m,
- E_m is independent view-specific Gaussian noise,
- the square-root residual term preserves valid covariance.

3.6 Crosstalk interpretation

If:

$$\gamma_m = 0$$

then:

$$f_m = E_m$$

The view has no shared latent component.

If:

$$\gamma_m = 1$$

the view uses the fitted template-level shared structure.

If:

$$0 < \gamma_m < 1$$

the view partially uses the shared structure.

If valid values above 1 are allowed, they amplify the shared structure beyond the fitted template level.

3.7 Valid crosstalk range

MimeSys requires:

$$I - \gamma_m B_m B_m^T$$

to remain positive semidefinite.

Therefore:

```
gamma_m <= 1 / sigma_max(B_m)^2
```

This is what crosstalk_max(template) computes.

3.8 Cross-view covariance induced by crosstalk

For two views m and m':

$$\mathrm{Cov}(f_m, f_{m'}) = \sqrt{\gamma_m \gamma_{m'}}\, B_m B_{m'}^T$$

So crosstalk directly scales cross-view latent covariance.

This is why increasing crosstalk generally increases cross-view feature correlation.

3.9 Simulate shared latent factors

For the factor simulator:

$$S_{\text{new}} \sim N(0, I_r)$$

Rows are samples and columns are shared latent factors.

If r = 0, there are no shared factors.

3.10 Simulate private view-specific factors

For each view:

$$E_m \sim N(0, I_{k_m})$$

Then:

$$f_m = \text{shared component} + \text{private component}$$

3.11 Generate latent normal-score features

For view m:

$$Z_m = f_m \Lambda_m^T + \varepsilon_m$$

For feature j:

$$\varepsilon_{ij} \sim N(0, \psi_j)$$

$$\psi_j = 1 - \|\Lambda_m[j,]\|^2$$

3.12 Map latent features to empirical scale

The latent normal-score values are converted to uniforms:

$$U_{mj} = \Phi(Z_{mj})$$

Then they are mapped through the empirical quantile function:

$$X_{mj} = Q_{mj}(U_{mj})$$

This preserves feature-specific skewness, discreteness, bounds, and empirical marginal behavior.

3.13 Copula dependence modes

For:

```
dependence = "empirical_copula"
dependence = "module_copula"
```

MimeSys uses a latent feature correlation matrix rather than the low-rank factor construction.

Crosstalk is applied by scaling cross-view blocks:

$$R_{ab} \leftarrow \sqrt{\gamma_a \gamma_b}\, R_{ab}$$

for a != b.

empirical_copula uses the empirical latent correlation more directly.

module_copula smooths correlations using within-view modules.

The role of crosstalk is unchanged: it scales cross-view dependence.

4. Microbiome Two-Stage Emission

4.1 Goal

Generate sparse, zero-inflated microbiome-like data while preserving dependence through the latent simulator.

This is part of null feature generation. It is not an outcome model.

4.2 When used

For example:

```
view.models = c(Microbiome = "two_part_copula")
```

Then the microbiome view uses a two-stage emission model.

4.3 Stage 1: presence/absence

For feature j, MimeSys generates a latent presence variable:

$$Z_{\text{presence},\,ij}$$

Then:

$$U_{\text{presence},\,ij} = \Phi(Z_{\text{presence},\,ij})$$

The empirical zero fraction from the template determines a threshold. Conceptually:

$$A_{ij} = 1 \text{ if } U_{\text{presence},\,ij} \text{ exceeds threshold}$$

$$0 \text{ otherwise}$$

If:

$$A_{ij} = 0$$

the emitted value is zero.

4.4 Stage 2: positive abundance

For present features:

$$A_{ij} = 1$$

MimeSys generates a second latent abundance variable:

$$Z_{\text{abundance},\,ij}$$

Then:

$$U_{\text{abundance},\,ij} = \Phi(Z_{\text{abundance},\,ij})$$

The positive abundance is:

$$V_{ij} = Q_j^{+}(U_{\text{abundance},\,ij})$$

4.5 Final microbiome value

The observed value is:

$$X_{ij} = A_{ij} V_{ij}$$

So:

- if A_ij = 0, then X_ij = 0,
- if A_ij = 1, then X_ij is positive and drawn from the empirical positive-value distribution.

4.6 Relationship to crosstalk

Crosstalk still acts upstream on the latent variables. The two-stage model changes how latent variables are emitted as observed features.

4.7 Relationship to outcome generation

After microbiome features are generated, any outcome mode can be used:

- LM,
- Friedman,
- Friedman2,
- SharedLatent with linear,
- SharedLatent with friedman,
- SharedLatent with friedman2.

5. Spike-In Coefficients

5.1 Goal

Create feature-level beta effects for observed-feature outcome modes.

These effects are used by:

- LM,
- Friedman,
- Friedman2.

They are also stored when using SharedLatent, but SharedLatent generates the final outcome from shared factors instead of observed-feature betas.

5.2 Number of nonzero effects

For each view:

$$n_{\mathrm{de},m} \sim \mathrm{Binomial}(p_m, \mathrm{de.prob}_m)$$

If n_de_m = 0, all effects in that view are zero.

5.3 Effect magnitudes

For each selected feature:

$$\mathrm{fac}_j \sim \mathrm{LogNormal}(\mathrm{de.facLoc}_m, \mathrm{de.facScale}_m)$$

The sign direction is initially sampled using de.downProb_m.

The beta value is:

$$\beta_j = \log_2(\text{effect factor})$$

Positive beta means increasing the feature increases the predictor. Negative beta means increasing the feature decreases the predictor.

5.4 Differential-effect alignment through de.align

Each feature has a stored:

$$\text{shared_loading}_j$$

This is its loading on the first shared latent direction.

If:

```
de.align = NULL
```

MimeSys uses independent spike-in positions and signs.

If:

```
de.align = a
```

with 0 <= a <= 1, then:

$$n_{\text{aligned}} = \text{round}(a\, n_{\text{de},m})$$

MimeSys samples those aligned features with probability proportional to:

$$\text{shared_loading}_j^2$$

For aligned features:

$$\text{sign}(\beta_j) = \text{sign}(\text{shared_loading}_j)$$

5.5 Interpretation of de.align

de.align controls whether outcome-relevant observed features point along the shared latent direction.

It is separate from crosstalk:

- crosstalk controls how views are correlated,
- de.align controls whether beta effects are aligned with that shared direction.

6. View-Level Signal Balancing

6.1 Goal

Control how much each view contributes to the additive observed-feature predictor.

The argument is:

```
signal.share
```

6.2 Raw view predictors

For each view:

$$\eta_m = X_m \beta_m$$

6.3 Normalizing requested shares

If:

```
signal.share = c(1, 1, 1)
```

MimeSys normalizes this to:

$$(1/3,\ 1/3,\ 1/3)$$

If named values are supplied, they are matched by view name.

6.4 Rescaling beta values

For each view, MimeSys computes:

$$\mathrm{Var}(\eta_m)$$

If this variance is positive, beta values for that view are multiplied by:

$$\sqrt{\frac{\text{target_share}_m}{\mathrm{Var}(\eta_m)}}$$

After this:

$$\mathrm{Var}(\eta_m) \approx \text{target_share}_m$$

6.5 Final additive linear predictor

The additive predictor is:

$$\eta_{\text{linear}} = \sum_m \eta_m$$

7. Outcome Generation Modes

7.1 ygen.mode = "LM"

The outcome signal is:

$$\eta = \eta_{\text{linear}} = \sum_m X_m \beta_m$$

This is an additive observed-feature outcome.

7.2 ygen.mode = "Friedman"

MimeSys selects five observed simulated features.

Candidate features are usually nonzero-beta features:

$$\text{candidate_index} = \{j : \beta_j \neq 0\}$$

If fewer than five are available, it falls back to all features.

The five selected observed features are rank-transformed:

$$U_j = \frac{\text{rank}(X_j)}{n+1}$$

Then:

$$\eta_{\text{Friedman}} =$$

$$10\sin(\pi U_1 U_2)$$

$$+20(U_3 - 0.5)^2$$

$$+10U_4$$

$$+5U_5$$

For Friedman:

$$\eta = \eta_{\text{Friedman}}$$

7.3 ygen.mode = "Friedman2"

MimeSys computes both:

$$\eta_{\text{linear}}$$

and:

$$\eta_{\text{Friedman}}$$

Then:

$$\eta = \eta_{\text{linear}} + \eta_{\text{Friedman}}$$

So this outcome combines additive observed-feature signal and nonlinear observed-feature signal.

7.4 ygen.mode = "SharedLatent"

The outcome signal is generated from shared latent factors instead of observed features.

During null generation, MimeSys simulates:

$$S_{\text{new}} \in \mathbb{R}^{n \times r}$$

Here:

- r is the number of shared latent factors in the template,
- rows are simulated samples,
- columns are shared latent axes.

Are all shared factors used?

By default:

```
shared.latent.n_factors = NULL
```

MimeSys uses all available shared latent factors:

$$K = r$$

If:

```
shared.latent.n_factors = K
```

MimeSys uses the first K shared factors:

$$S_K = [S_1, \ldots, S_K]$$

If K > r, MimeSys stops with an error.

Why multiple shared factors matter

Using only one latent factor can be limiting because cross-omics biology may not be one-dimensional.

Using multiple shared factors allows the outcome to depend on several shared biological axes, for example:

- immune activation,
- metabolic shift,
- proteomic stress response,
- inflammatory pathway activity,
- microbiome-derived metabolite axis.

Shared latent linear outcome

For:

```
shared.latent.fun = "linear"
```

MimeSys standardizes selected shared factors:

$$S_{\text{scaled}} = \text{scale}(S_K)$$

Then computes:

$$\eta_{\text{linear, shared}} = \frac{\text{rowSums}(S_{\text{scaled}})}{\sqrt{K}}$$

The division by sqrt(K) prevents variance from increasing just because more factors are included.

The final predictor is:

$$\eta = \text{scale}(\eta_{\text{linear, shared}})$$

Shared latent Friedman outcome

For:

```
shared.latent.fun = "friedman"
```

MimeSys requires:

```
K >= 5
```

It takes the first five selected shared factors and rank-transforms them:

$$U_j = \frac{\text{rank}(S_j)}{n+1}, \quad j = 1, \ldots, 5$$

Then:

$$\eta_{\text{shared, Friedman}} =$$

$$10\sin(\pi U_1 U_2)$$

$$+20(U_3 - 0.5)^2$$

$$+10U_4$$

$$+5U_5$$

The final predictor is:

$$\eta = \text{scale}(\eta_{\text{shared, Friedman}})$$

Shared latent Friedman2 outcome

For:

```
shared.latent.fun = "friedman2"
```

MimeSys computes:

$$\eta_{\text{linear, shared}}$$

and:

$$\eta_{\text{shared, Friedman}}$$

Then:

$$\eta = \text{scale}(\eta_{\text{linear, shared}} + \eta_{\text{shared, Friedman}})$$

Role of crosstalk in shared-latent outcomes

In SharedLatent, the true outcome signal is:

$$Y_{\text{signal}} = f(S_{\text{new}})$$

Crosstalk controls how strongly S_new is visible in each observed omics layer.

Low crosstalk means the outcome-driving shared factors may be weakly represented in observed features.

High crosstalk means those shared factors are more strongly reflected in every layer.

Very high crosstalk can make layers redundant, which may not help fusion.

8. Final Outcome Types

Once eta is computed, MimeSys generates the actual outcome.

8.1 Continuous outcome

For:

```
outcome.type = "continuous"
```

MimeSys computes:

$$\sigma^2 = \frac{\text{Var}(\eta)}{\text{snr}}$$

Then:

$$Y_i = \eta_i + \varepsilon_i$$

$$\varepsilon_i \sim N(0, \sigma^2)$$

Large snr means lower noise. Small snr means higher noise.

8.2 Binary outcome

For:

```
outcome.type = "binary"
```

MimeSys uses:

$$\Pr(Y_i = 1) = \text{logit}^{-1}(\eta_i)$$

Then:

$$Y_i \sim \text{Bernoulli}(\text{logit}^{-1}(\eta_i))$$

8.3 Survival outcome

For:

```
outcome.type = "survival"
```

MimeSys computes:

$$h_i = \text{surv.hscale} \exp(\eta_i)$$

Then:

$$T_i \sim \text{Exponential}(\text{rate} = h_i)$$

Censoring is:

$$C_i \sim \text{Uniform}(\text{cens.lower}, \text{cens.upper})$$

Observed output is:

$$\text{time}_i = \min(T_i, C_i)$$

```
status_i = 1 if T_i <= C_i, 0 otherwise
```

9. Train/Test Split

MimeSys samples:

$$\text{round}(\text{nsample} \times p_{\text{train}})$$

samples for training.

It then subsets:

- feature table,
- sample metadata,
- factor scores,
- shared latent factors,
- outcome metadata.

The outcome is generated once before splitting.

10. Crosstalk Calibration

calibrate_crosstalk() is used when the user wants to choose crosstalk by reconstruction of empirical cross-view correlation.

The process is:

1. Choose a grid of candidate crosstalk values.
2. Simulate null datasets at each value.
3. Compute cross-view feature-correlation blocks.
4. Compare simulated blocks with empirical/template blocks.
5. Compute reconstruction error, usually rescaled Frobenius-type error.
6. Select the crosstalk value with the lowest average reconstruction error.

This selects crosstalk for feature-structure reconstruction, not for predictive performance.

11. Summary of Controls

rank:

- within-view factor rank,
- if NULL, chosen by variance explained,
- not 2 log(p) by default.

shared.rank:

- number of shared cross-view latent factors,
- if NULL, chosen by permutation-based parallel analysis,
- if numeric, forced to that value.

crosstalk:

- strength of cross-view feature dependence,
- 0 removes shared latent dependence,
- 1 uses fitted template shared structure.

de.align:

- aligns beta effects with shared latent loading direction.

signal.share:

- controls how much each view contributes to additive observed-feature signal.

friedman.share:

- controls which views supply the five observed features used in Friedman/Friedman2.

shared.latent.n_factors:

- controls how many shared latent factors are used for SharedLatent outcomes,
- NULL means use all available shared factors.

shared.latent.fun:

- "linear": linear shared-latent score,
- "friedman": nonlinear Friedman function on shared factors,
- "friedman2": linear plus nonlinear shared-latent signal.

12. Recommended Controlled Shared-Latent Setup

For controlled nonlinear shared-latent validation:

```
template <- fit_template(
  views,
  rank = 10,
  shared.rank = 5
)

sim <- gen_simmba(
  nsample = 200,
  template = template,
  p = c(100, 100, 100),
  crosstalk = 0.75,
  ygen.mode = "SharedLatent",
  shared.latent.fun = "friedman2",
  shared.latent.n_factors = 5,
  snr = 0.5
)
```

This makes the shared latent structure explicit:

- exactly five shared factors are fitted,
- exactly five shared factors are used for the outcome,
- the outcome contains both linear and nonlinear shared-latent signal,
- crosstalk controls how strongly that outcome-driving latent structure appears in each observed omics layer.